

Assignment 1: Text Processing Fundamentals using MapReduce

Chi-Square Term Selection over 22 Amazon Review Categories

Group 48 – Data-Intensive Computing, TU Wien, SS 2026

Aaradhaya Arora (12534787) Ekaterina Dobrynina (12332276) Gopinath Hariharasudhan (52202173)

Serhii Slutu (12537831) Sofia Zubkova (12534780)

April 25, 2026

AI disclosure. We used Anthropic’s Claude Opus 4.7 as a support tool while working on this assignment. Its use was limited to text polishing (grammar, wording) and to debugging help when we were stuck. The pipeline design, the choice of `<key,value>` pairs, the chi-square derivation and all implementation decisions were made by us; the model did not generate ideas or solutions, it was used purely as assistance.

1 Introduction

The goal of this assignment is to pick, for each of the 22 Amazon review categories, the terms that are most useful for telling one category apart from the others – a typical feature selection step before any real text classifier is trained. We use the chi-square test for that, because it only needs document-frequency counts and fits MapReduce nicely.

The dataset is big enough that we cannot simply load it into memory: the course provides a small development set (`reviews_devset.json`, about 78 800 reviews) and a full 56 GB version on HDFS. We developed and tested everything on the development set and wrote the code so that running it on the full corpus is just a matter of changing the input path.

In the rest of this report we describe the structure of the pipeline, what each mapper and reducer actually does (including the `<key,value>` pairs that flow between them) and what the output on the development set looks like.

2 Problem Overview

Input. Each line of the input file is a JSON document. Out of all the fields we only look at two: `reviewText` (the text to process) and `category` (the class label).

Pre-processing. Before counting anything we have to clean up the text. The steps are the ones described in the assignment:

- lowercase the review text,
- split on whitespace, tabs, digits and the characters

`( ) [ ] { } . ! ? , ; : + = - _ " ' ~ # @ & * % $ \ /`

plus `€` and `§`,

- drop tokens that are in `stopwords.txt` or that have length 1.

Because chi-square cares about *how many documents* contain a term in a given category (not how many times it occurs), we also de-duplicate the tokens of a review before counting them. So from every review we keep a *set* of unigrams.

Chi-square. For a term t and a category c we build the usual 2×2 contingency table:

$$\begin{array}{c|cc} & t \in d & t \notin d \\ \hline d \in c & A & C \\ d \notin c & B & D \end{array} \quad N = A + B + C + D.$$

Here A is the number of documents of category c that contain t . If we also know n_c (the size of category c) and $f_t = A + B$ (how many documents in total contain t), we can fill in the other cells directly:

$$B = f_t - A, \quad C = n_c - A, \quad D = N - n_c - B.$$

With the standard simplified formula,

$$\chi^2(t, c) = \frac{N(AD - BC)^2}{(A + B)(A + C)(B + D)(C + D)}, \quad (1)$$

we can compute χ^2 from just four numbers: A , f_t , n_c and N . This is convenient, because the first MapReduce job is going to produce exactly those.

Output. What we have to produce is one text file, `output.txt`, with:

- one line per category, sorted alphabetically, in the format `<category> t1: $\chi^2_1$  ... t75: $\chi^2_{75}$` ;
- one final line with the merged dictionary, i.e. all terms that appear in any of the 22 top-75 lists, sorted alphabetically and space-separated.

3 Methodology and Approach

3.1 Pipeline overview

We split the work into two MapReduce jobs. The first one counts; the second one computes chi-square, picks the top-75 per category and builds the merged dictionary. A tiny text file (`meta.tsv`, under one kilobyte) is produced by the driver between the two jobs and carries the category sizes n_c and the global count N to the second job.

Figure 1 shows the whole pipeline with the `<key,value>` pair on every arrow, which is the design contract we tried to keep as explicit as possible.

3.2 Stage 1 – counting

The first job (`chi_count.py`) reads the JSON review lines and emits three kinds of records:

Record	Meaning	Useful for
<code>((“t”, t, c), 1)</code>	document contains term t in category c	$A(t, c)$
<code>((“c”, c), 1)</code>	document belongs to category c	n_c
<code>((“n”,), 1)</code>	one more document seen	N

In the mapper we keep per-task Python dictionaries (`_buf_tc`, `_buf_c` and a counter `_buf_n`) and only emit the records in `mapper_final()`. This is the in-mapper combining pattern and it cuts the amount of data going into the shuffle a lot, because a term that appears in many reviews of the same split is aggregated locally first. A normal combiner with the same logic as the reducer sits on top of it. The reducer simply sums the partial counts for each key.

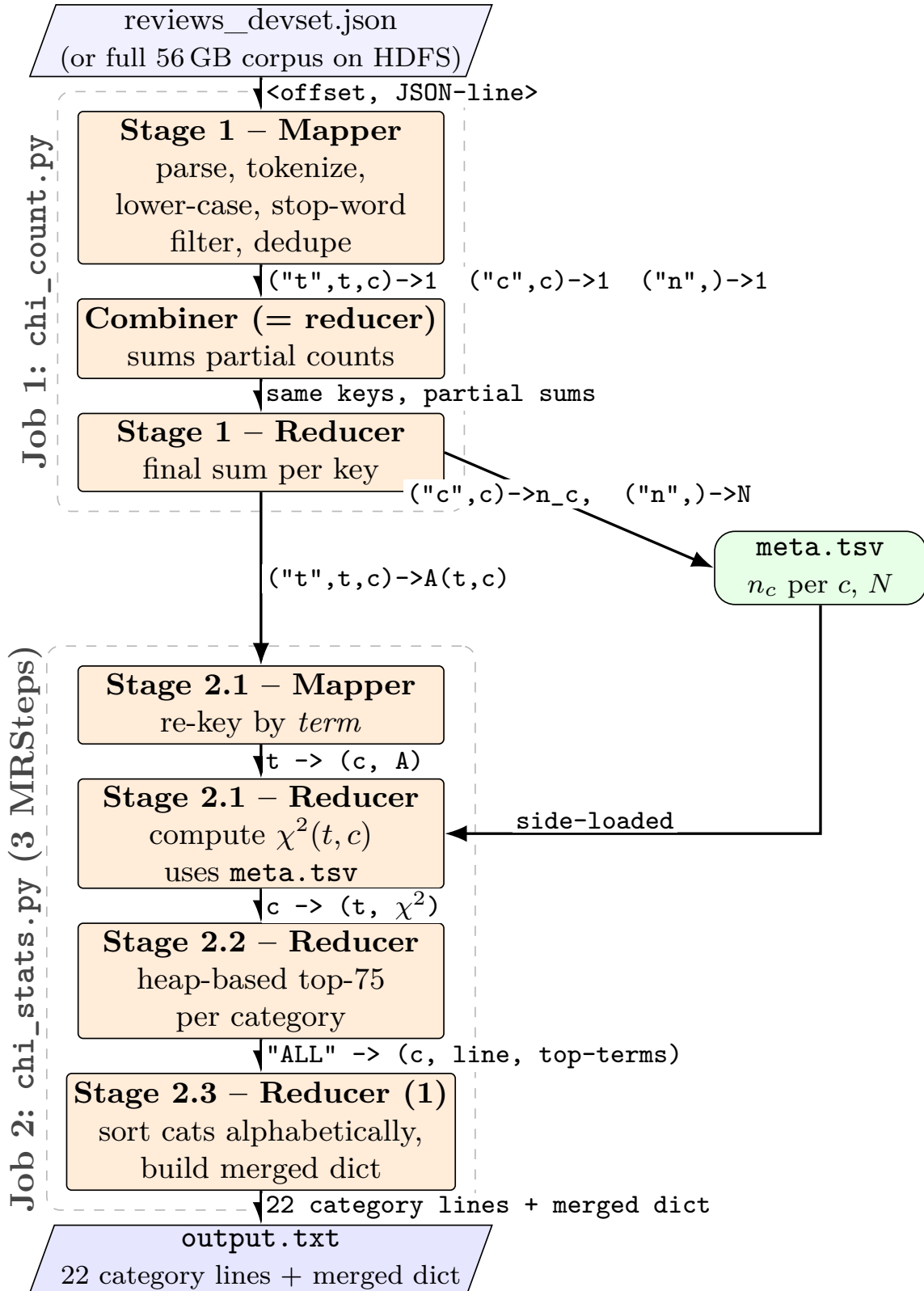


Figure 1: End-to-end chi-square pipeline. Every arrow is labelled with the $\langle \text{key}, \text{value} \rangle$ pair that flows through it. Orange rectangles are MapReduce tasks, the blue parallelograms are data sets on disk, and the green rounded box is the small meta file that the driver passes from the first job to the second.

3.3 Between the two jobs – meta.tsv

The driver (`run.sh` on the cluster, `run_local.py` for local testing) reads the Stage-1 output once, keeps only the "c" and "n" records and writes a tiny TSV file:

Listing 1: meta.tsv (excerpt)

n	78829	
c	Apps_for_Android	2638
c	Automotive	1374
...		

The term-level records stay on HDFS; only this small file is shipped to the Stage-2 tasks via mrjob's `-file` mechanism. We chose this over the two obvious alternatives – either replicating n_c and N on every term record (which would blow up the shuffle) or sending everything through a single reducer (which would kill parallelism).

3.4 Stage 2 – chi-square, top-75 and merged dictionary

The second job (`chi_stats.py`) has three MRSteps:

1. **Regroup by term.** The mapper re-emits each count as `(term, (cat, A))`. The "c" / "n" records from Stage 1 are simply ignored here, because that information is already in `meta.tsv` (loaded in `mapper_init` / `reducer_init`). The reducer then sees all `(cat, A)` pairs for a single term in one go, which is exactly what we need: we sum them to get f_t , look up n_c and N in the meta file, derive B , C , D and emit `(cat, (term,  $\chi^2$ ))` using Eq. (1). If the denominator is zero (this happens only when a term appears exclusively in one category) we just skip that pair.
2. **Top-75 per category.** The second reducer receives all `(term,  $\chi^2$ )` pairs of a given category and keeps only the 75 best using a bounded min-heap. We emit the whole formatted line for the category as one record.
3. **Final assembly.** The last step runs with a single reducer (forced via `mapreduce.job.reduces=1`). It collects the 22 category lines, sorts them alphabetically, prints them, and then prints the merged dictionary. At that point the data is tiny – 22 short lines – so a single reducer is absolutely fine.

3.5 Design choices and why we made them

- **Document frequency, not raw frequency.** Deduplicating the tokens of each review keeps the χ^2 formula correct and also keeps the intermediate key space small.
- **In-mapper combining + combiner.** Together they greatly reduce the amount of data that actually goes through the shuffle, which is usually the slowest part of a MapReduce job.
- **Meta broadcast via a small side file.** This keeps the reducer parallelism intact while still giving every reducer access to the global totals it needs.
- **Heap for top- K .** $O(V_c \log K)$ per category is simpler and faster than sorting and then slicing.
- **Single reducer only at the very end.** We only force the sequential bottleneck where the amount of data is already tiny.
- **Parameterised input path.** Both drivers take the input path as their first argument, so switching from the dev set to the full 56 GB corpus is a one-liner.

3.6 Results on the development set

We tested the whole pipeline on `reviews_devset.json` (78 829 reviews, 22 categories). End-to-end it runs in about 20 seconds on a normal laptop using `mrjob`'s inline runner, which is what we used during development. The resulting `output.txt` has exactly the expected shape: 22 category lines of 75 tokens each, plus one final line with the merged dictionary (1 463 unique terms in our case).

The terms that come out on top look sensible for each category, for instance *music* for CDS__AND__VINYL ($\chi^2 \approx 13\,083$), *dvd* for MOVIES__AND__TV ($\approx 8\,747$), *reading* for BOOK ($\approx 6\,185$), *iphone* for CELL__PHONES__AND__ACCESSORIE ($\approx 4\,184$), *diaper* for BABY ($\approx 2\,430$), and *cat* for PET__SUPPLIE ($\approx 4\,345$). That matches what we expected from the category names and gave us confidence that the implementation is doing the right thing.

4 Conclusions

We built a two-job MapReduce pipeline that turns the Amazon review corpus into a small, chi-square based vocabulary. Stage 1 handles the heavy counting, Stage 2 does the statistics and the top- K selection, and a tiny meta file in between carries the global totals so that Stage 2 can stay parallel. On the development set the whole thing runs in about 20 seconds and produces exactly the output format requested by the assignment. The same code runs on the cluster without modification – only the input path changes – which is what makes the transition to the full 56 GB dataset painless for the final evaluation.